c++ 引用传参和指针传参的区别

原创 wrdasj 于 2019-05-26 19:32:47 发布 阅读量1.4w 收藏 135 点赞数 30
分类专栏： 编程 文章标签： c++ 引用传参和指针传参的区别

概念

- 指针从本质上讲是一个变量，变量的值是另一个变量的地址，指针在逻辑上是独立的，它可以被改变的，包括指针变量的值（所指向的地址）和指针变量的值对应的内存中的数据（所指向地址中所存放的数据）。
- 引用从本质上讲是一个别名，是另一个变量的同义词，它在逻辑上不是独立的，它的存在具有依附性，所以引用必须在一开始就被初始化（先有这个变量，这个实物，这个实物才能有别名），而且其引用的对象在其整个生命周期中不能被改变，即自始至终只能依附于同一个变量（初始化的时候代表的是谁的别名，就一直是谁的别名，不能变）。

区别

指针	变量，独立，可变，可空，替身，无类型检查
引用	别名，依赖，不变，非空，本体，有类型检查

- 指针参数传递本质上是值传递，它所传递的是一个地址值。值传递过程中，被调 函数 的形式参数作为被调函数的局部变量处理，会在栈中开辟内存空间以存放由主调函数传递进来的实参值，从而形成了实参的一个副本（替身）。值传递的特点是，被调函数对形式参数的任何操作都是作为局部变量进行的，不会影响主调函数的实参变量的值（形参指针变了，实参指针不会变）。
- 引用参数传递过程中，被调函数的形式参数也作为局部变量在栈中开辟了内存空间，但是这时存放的是由主调函数放进来的实参变量的地址。被调函数对形参（本体）的任何操作都被处理成间接寻址，即通过栈中存放的地址访问主调函数中的实参变量（根据别名找到主调函数中的本体）。因此，被调函数对形参的任何操作都会影响主调函数中的实参变量。
- 引用传递和指针传递是不同的，虽然他们都是在被调函数栈空间上的一个局部变量，但是任何对于引用参数的处理都会通过一个间接寻址的方式操作到主调函数中的相关变量。而对于指针传递的参数，如果改变被调函数中的指针地址，它将应用不到主调函数的相关变量。如果想通过指针参数传递来改变主调函数中的相关变量（地址），那就得使用指向指针的指针或者指针引用。

- 从编译的角度来讲，程序在编译时分别将指针和引用添加到符号表上，符号表中记录的是变量名及变量所对应地址。指针变量在符号表上对应的地址值为指针变量的地址值，而引用在符号表上对应的地址值为引用对象的地址值（与实参名字不同，地址相同）。符号表生成之后就不会再改，因此指针可以改变其指向的对象（指针变量中的值可以改），而引用对象则不能修改。

以上内容来源网络

如何使用

> 如果我们编程只是为了做一些简单的工作，不想在技术上有什么质的突破，那么使用指针就够了。这样做的问题就是，你写的代码很容易出bug。可是，如果我们想要对c++有更深的了解，那么一定要区分指针传参与引用传参的区别。

使用指针传递参数，是为了避免拷贝副本，从而提高程序执行效率，大所数初学者（也包括我）都习惯使用指针传递参数，因为还没有建立起引用的概念，我们以为指针就是引用，殊不知，引用的内涵比指针更丰富。因为用的少，所以对引用的概念懵懵懂懂。其实有经验的程序员在传递参数时很少使用指针。下面通过代码来说明我们为什么要使用引用传参，放弃指针传参。

- **首先定义一个结构体**

```
struct ST
{
public:
    int num;
    string name;
    ST(int num, string name)
    {
        this->num = num;
        this->name = name;
    }
    void putInfo()
    {
        cout << "学号: " << num << endl;
        cout << "姓名: " << name << endl;
    }
};
```

1.使用指针传参

```
void dealProcess(ST* st)
{
    ST *newST =new ST(320, "小明");
    st = newST;//为临时指针变量赋予了新值，已和输入脱离了联系。
}

int main()
{
    int num = 1;
    ST st1(310, "小红");
    dealProcess(&st1);
    st1.putInfo();
    system("pause");
    return 0;
}
```

- 输出结果

> 使用指针传参，想返回处理后的结果，但是我们得到的结果却和预想的不一致。因为在处理的过程中，形参的地址发生了变化，和输入脱离了联系。

- 正确的写法：

```
void dealProcess(ST* st)
{

```

```
    ST *newST =new ST(320, "小明");
    *st = *newST;
}
```

- 严谨的写法（但不简洁）

```
void dealProcess(ST* const st)//指定形参为常量指针，不能修改值
{
    ST *newST =new ST(320, "小明");
    *st=*newsT;
    st = newST;    //编译错误，不允许修改常量
}
```

`void dealProcess(ST* const st)`等价于`void dealProcess(ST* st const)`

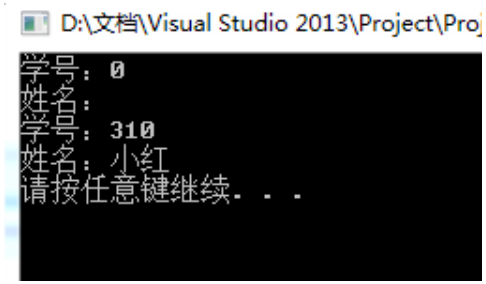
2.既严谨又简洁的写法，使用引用

```
void dealProcess(ST& st)
{
    ST newST(320, "小明");
    st = newST;
    &st=&newsT;//编译错误，不允许改变输入的地址
}
int main()
{
    int num = 1;
    ST st1(310, "小红");
    dealProcess(st1);
    st1.putInfo();
    system("pause");
    return 0;
}
```

3.如果仅仅是使用输入的值，应该限定输入为常量

- **首选是使用引用**

```
void dealProcess(const ST& st)
{
    ST newST(0,"");
    newST.putInfo();
    //输入的值不允许被改变
    st = newST;  //编译错误，只允许使用输入,不允许改变输入的值
    &st = &newsT;//编译错误，不允许改变输入的地址
    //使用输入的值,
    newST = st;
    newST.putInfo();
}
int main()
{
    int num = 1;
    ST st1(310, "小红");
    dealProcess(st1);
    system("pause");
    return 0;
}
```



- **如果一定要使用指针，等价代码如下**

```
void dealProcess(const ST* const st)
{
    ST newST(0,"");
    newST.putInfo();
    //使用输入的值
    newST = *st;
    newST.putInfo();
```

```
}
int main()
{
    int num = 1;
    ST st1(310, "小红");
    dealProcess(&st1);
    system("pause");
    return 0;
}
```

c++ 引用和指针传参的区别以及优缺点

鱼弦
技术推广行业 从业人员

+ 关注

来自专栏 • 摸鱼之技术道 >

11 人赞同了该文章 ›

在 C++ 中，函数参数的传递方式有值传递、指针传递和引用传递⁺三种方式。其中，指针传递和引用传递都可以实现在函数内部修改传入的变量值，但二者有以下区别：

1. 语法上的区别：指针传递使用指针类型的形参，传递的是指针所指向的变量地址；引用传递使用引用类型的形参，传递的是实参的别名。
2. 操作上的区别：对指针变量进行解引用⁺（即 * 操作符），可以访问指针所指向的变量的值；对引用变量⁺进行操作，则直接操作实参的值。
3. 空指针和空引用的区别：指针可以为 NULL 或 nullptr，表示指向空内存地址⁺；而引用必须绑定在某个对象上，不可能为空。

指针传递的优点是可以显式地表明在函数中会修改传入的实参的值，缺点是需要对指针进行解引用，使用时需要注意指针是否为空，容易产生野指针⁺和空指针异常等问题。引用传递的优点是可以避免指针操作的复杂性，使用方便，且不存在指针为空的问题，但也有一定的缺点，比如需要使用 & 操作符来取实参的别名，有时可能不太直观。

在 C++ 中，引用和指针都可以用于传递函数⁺参数。它们都是传递地址，而不是传递值，可以实现函数中对参数的修改。但是它们之间还是存在一些区别，下面分别介绍一下：

1. 语法形式：

```
int x = 1;
int& ref = x; // 声明一个引用
int* ptr = &x; // 声明一个指针
```

1. 空值处理：

引用必须初始化，且不能为空。指针可以不初始化，可以为空。

```
int x = 1;
int& ref = x; // 引用必须初始化
int* ptr; // 指针可以不初始化
ptr = &x; // 指针可以在后面指向某个地址
```

1. 操作符号：

对于引用，可以像使用变量一样使用引用，不需要使用额外的操作符。对于指针，需要使用 `*` 符号来解引用，即访问指针指向的值。

```
int x = 1;
int& ref = x;
int* ptr = &x;
ref = 2; // 使用引用修改变量
*ptr = 3; // 使用指针修改变量
```

1. 引用和指针的优缺点：

引用的优点在于其语法简单，使用起来方便，不需要像指针那样使用额外的操作符。同时，引用在声明时必须进行初始化，可以避免空指针引起的问题。但是引用也有缺点，比如不能修改引用的目标对象，因为引用的目标对象是常量。

指针的优点在于可以指向空地址，可以在需要时动态地分配内存。同时，指针可以进行运算，可以方便地进行数组操作。但是指针也有缺点，比如需要使用额外的操作符，语法相对较复杂，而且容易引起空指针和野指针的问题。

引用和指针的语法形式有所不同。引用使用 `&` 符号来声明，指针使用 `*` 符号来声明。

因此，选择使用指针传递还是引用传递，应该根据具体的应用场景和需求来进行选择。通常来说，对于简单类型的参数，可以使用值传递；对于需要修改实参的情况，可以使用指针传递或引用传递；对于需要在函数中创建新对象的情况，应该使用指针传递。

C++函数返回值是引用&

原创 晓_X 于 2025-03-10 18:13:11 发布 阅读量310 收藏 7 点赞数 4 版权
文章标签： c++ 开发语言 qt 学习

目录

1、C++ 返回引用的核心要点
2、返回引用的作用
3. 返回引用的注意事项
4、示例

在 C++ 中，函数可以返回引用（&），这是一种非常强大的特性。返回引用的函数可以直接操作调用者提供的对象，避免了不必要的拷贝，同时也可以用于实现 链式调用 等高级功能。以下是关于 C++ 引用作为返回值的详细说明。

并且可以通过 调用函数 名称进行赋值，在 C++ 中，**能直接调用函数名称赋值**是因为函数返回的是 **引用（&）**，而引用本质上是一个别名，它直接绑定到某个变量或对象上。因此，通过引用可以直接修改原对象的值。

1、C++ 返回引用的核心要点

```
int& functionName(parameters);//int为举例
```

`int&` 表示返回类型是一个引用。

`functionName` 是函数名。

`parameters` 是函数参数。

2、返回引用的作用

（1）避免拷贝：

返回引用可以避免返回对象的拷贝，提高性能。

特别是对于大型对象（如容器、类对象），返回引用可以显著减少开销。

（2）直接修改调用者的对象：

返回引用允许函数直接修改调用者提供的对象。

（3）支持链式调用：

返回引用可以用于实现链式调用（如 std::cout 的 << 操作符）。

3. 返回引用的注意事项

（1）不能返回局部变量的引用

局部变量在函数结束时会被销毁，返回其引用会导致未定义行为。

例如：

```
int& badFunction() {
    int x = 10;
    return x; // 错误：返回局部变量的引用
}
```

（2）可以返回静态变量或全局变量的引用：

静态变量和全局变量的生命周期贯穿整个程序，可以安全地返回其引用。

```
int& goodFunction() {
    static int x = 10;
    return x; // 正确：返回静态变量的引用
}
```

（3）可以返回参数或成员变量的引用：

如果参数或成员变量的生命周期足够长，可以返回其引用。

```
int& getElement(std::vector<int>& vec, int index) {
```

```
    return vec[index]; // 返回 vector 中元素的引用
}
```

4、示例

（1）返回数组元素的引用，直接用=

```
#include <iostream>

int& getElement(int arr[], int index) {
    return arr[index]; // 返回数组元素的引用
}

int main() {
    int arr[] = {10, 20, 30};
    getElement(arr, 1) = 100; // 修改数组元素
    std::cout << arr[1] << std::endl; // 输出: 100
    return 0;
}
```

（2）修改成员变量

```
#include <iostream>

class MyClass {
public:
    int value;

    int& getValue() {
        return value; // 返回成员变量的引用
    }
};

int main() {
    MyClass obj;
    obj.getValue() = 42; // 修改成员变量
```

```
    std::cout << obj.value << std::endl; // 输出: 42
    return 0;
}
```

常见的容器类的访问函数：

`std::vector` 的 `operator[]` 返回元素的引用，允许直接修改元素

```
std::vector<int> vec = {1, 2, 3};
vec[0] = 10; // 修改 vector 的第一个元素
```

3、链式调用

```
#include <iostream>

class Counter {
public:
    int count = 0;

    // 返回当前对象的引用
    Counter& increment() {
        count++;
        return *this;
    }
};

int main() {
    Counter c;
    c.increment().increment(); // 链式调用
    std::cout << c.count;      // 输出 2
    return 0;
}
```